PrivaCred: Privacy-Preserving Attribute-Based Authorisation Using BBS+ Selective Disclosure and zk-SNARK Predicate Proofs in W3C Verifiable Credentials

Damire Dominic Zalyatov
Queen Mary University of London
London, United Kingdom

Abstract—Verifying parties routinely receive far more information about an individual than they need. A centralised KYC check hands over the holder's whole stored record. Where OAuth is used, the stable subject identifier it returns can be correlated across every service that accepts it. Sign-In With Ethereum goes further still, since the holder's permanent wallet address travels with every login. This paper describes the design and implementation of PrivaCred, a system that uses BBS+ selective disclosure, zk-SNARK predicate proofs (selectable at run-time between Groth16 and PLONK), and pairwise pseudonymous DIDs within W3C Verifiable Credentials to prove a set of eligibility predicates without revealing the underlying values of the holder's attributes. A single Circom circuit verifies a Poseidon commitment over all credential attributes and proves three predicates in the same proof. That commitment is also what binds the two curves together. BBS+ signs it on BLS12-381 and the SNARK takes it as a public input on BN254, and the hash equality linking them holds under Groth16 and under PLONK alike. Predicate proofs then sit at presentation level in the W3C JSON layout defined here, which keeps what is proved apart from what is presented. In nine experiments, PrivaCred released no holder attribute values, admitted no cross-verifier correlation by shared identifier or value on the channels measured, bounded to the pool size, and verified the predicates offline under both proving systems. The correlation result is measured at the protocol level, and does not extend to network- or transport-layer channels. The compressed proof object of the Groth16 system is approximately 128 bytes long (three group elements on BN254). The serialised presentation envelope is a few kilobytes long, larger under the PLONK proving system. In contrast, the baselines release between one and seven attribute values and one or more stable correlation handles per session.

Index Terms—verifiable credentials, BBS+ signatures, zero-knowledge proofs, selective disclosure, decentralised identity, privacy-preserving authentication

I. Introduction

CURRENTLY used digital identity systems release more information about the holder of a digital identity than is needed to verify a single, narrow claim of fact. A KYC provider passes those attributes on, and the parties receiving them go on to verify claims of their own. With OpenID Connect, every login returns a stable subject identifier alongside an email address (Sakimura et al., 2014), and formal analysis has since found further correlation and leakage risks in those flows (Fett et al., 2016). The bluntest case is Sign-In With Ethereum (SIWE) (Chang et al., 2021), which hands over a permanent wallet address. Repeated release of information across verifiers defeats unlinkability in the sense of Pfitzmann and Hansen (2010) and is a precondition for de-anonymisation attacks (Narayanan and Shmatikov, 2008). This dissertation presents a system that discloses zero attribute values of the holder.[1]

Anonymous-credential research has produced privacy-preserving primitives for over two decades (Camenisch and Lysyanskaya, 2001; Camenisch and Herreweghen, 2002; Au et al., 2006), and the W3C Verifiable Credential Data Model v2.0 (Sporny, Thibodeau Jr., Herman, Cohen and Jones, 2025) provides a deployment-ready container. Yet the intersection of full attribute hiding (no values disclosed at all, only proofs of predicates over them) and W3C VC alignment remains sparsely explored in the academic literature, although recent industrial systems address adjacent settings: zk-creds (Rosenberg et al., 2023) attests to government-issued passports rather than BBS+ credentials, Microsoft's Crescent (Paquin et al., 2024) and Google's ECDSA-based anonymous credentials (Frigo and abhi shelat, 2026) target JWT and ISO-mdoc formats, and Dock/Truvera supports Circom predicates over BBS+ attributes via LegoGroth16 on BLS12-381, outside the W3C Data Integrity presentation format (Dock Labs, 2024). No prior open implementation binds BBS+ credentials to the mainstream circom/snarkjs Groth16 toolchain on BN254 within standard W3C Verifiable Presentations.

Aim and objectives. The aim of this project is to design and build a verifiable-credential system in which a holder proves, without revealing any holder attribute value, that they satisfy given eligibility predicates. The

MSc dissertation, School of Electronic Engineering and Computer Science, Queen Mary University of London, 2025/26 academic year.

[1] An end-to-end browser demonstrator (real BBS+/SNARK proofs, not mocks) ships in packages/demo-server; deployment guide in its README.

objectives are to: (O1) design a construction that binds BBS+ credential authenticity to SNARK predicate proofs over the holder's attributes (Section IV); (O2) implement the construction from (O1) end-to-end on a mainstream, auditable toolchain (Section V); (O3) evaluate the disclosure, unlinkability, performance and robustness of the implementation against three deployed baselines: full disclosure of all the holder's attributes, a stable subject identifier plus email address, and Sign-In With Ethereum (Section VI); and (O4) characterise the security properties of the design and implementation and state the limitations of the resulting system (Appendix A).

Contribution 1: A single Circom circuit, PrivaCredFullProof, verifies a Poseidon commitment over all credential attributes. The same proof also shows that an age range predicate, a boolean equality and a hash equality hold together, under either Groth16 or PLONK.

Contribution 2: BBS+ operates on BLS12-381 and the SNARK on BN254, two curves with different scalar fields. The Poseidon commitment bridges them. BBS+ signs it as its sole message and the SNARK takes it as a public input, so a single field-element equality carries the whole binding. Groth16 and PLONK both satisfy that equality.

Contribution 3: In the W3C VP JSON layout defined here, predicate proofs sit at presentation level, outside the credential's signature proof. That keeps two authorities apart. BBS+ proves the credential is authentic. The SNARK proves the presentation conditions hold.

We evaluate PrivaCred against three baselines (centralised KYC, OAuth 2.0/OIDC, SIWE) over nine experiments measuring disclosure, linkability under collusion, performance, revocation, replay, predicate correctness, payload audit, predicate-compiler cost, and concurrency.

II. Background

BBS+ signatures, introduced in Au et al. (2006) as a variant of the Boneh–Boyen–Shacham group signature (Boneh et al., 2004), sign a vector of messages $(m_1, \ldots, m_L)$ under a single tag. The basic BBS signatures were subsequently modified to enable selective disclosure of the messages: a holder of a BBS+ signature can prove disclosure of any subset of the messages while keeping the others secret, and the individual proofs are unlinkable, so different proofs can be produced in different presentations. The W3C BBS Cryptosuite draft (Bernstein and Sporny, 2026) and the corresponding IRTF draft (Looker et al., 2026) standardise the modern BBS scheme in the revised form of Tessaro and Zhu (2023), over the BLS12-381 pairing-friendly curve, with a JSON-LD integrity proof type that fits inside a Verifiable Credential.

A zero-knowledge proof (Goldwasser et al., 1989) allows one party to prove to another that certain statements are true without revealing anything more about them. Groth16 (Groth, 2016) builds such a proof from three group elements on the BN254 curve. Compressed, that is about 128 bytes, and checking it takes constant time. The cost is a trusted setup, which must be run again for every new circuit. An alternative is PLONK (Gabizon et al., 2019): a single universal trusted setup for all circuits, which in exchange increases proof size and decreases proving speed. We instantiate the predicate layer with both proving systems and compare them empirically in Section VI. Circuits in this work are written in Circom against the circomlib primitive library, including the ZK-friendly Poseidon hash and an arithmetic comparator.

W3C Verifiable Credentials (Sporny, Thibodeau Jr., Herman, Cohen and Jones, 2025) include a JSON-LD container holding the claims about a subject issued by an issuer, together with a signature from the issuer. In combination with Decentralised Identifiers (Sporny et al., 2022), they hold long-lived subject and issuer references resolvable without a central registry. The Bitstring Status List specification (Sporny, Longley, Prorock and Alkhraishi, 2025) encodes revocation information in a public bitstring credential that can be cached offline by the verifiers.

Selective-disclosure JWTs (SD-JWT) (Fett et al., 2025) and the ETSI EU-Wallet analysis (ETSI, 2025) cover salted-hash disclosure, whose disclosed payload is the cleartext attribute; PrivaCred discloses only a Poseidon commitment plus predicate proofs.

III. Related Work

Idemix, IRMA and Coconut. Idemix (Camenisch and Herreweghen, 2002) is an anonymous-credential system built on CL signatures, and it does support predicate proofs. It predates the W3C VC data model, however, and rests on an RSA-based primitive that the modern BBS suite does not carry. The IRMA wallet (Alpár et al., 2017) is implemented on top of Idemix and widely deployed for attribute-based authentication, but is designed for a different presentation format than Verifiable Credentials. Coconut (Sonnino et al., 2019) enables threshold issuance of selective-disclosure credentials, but likewise sits outside the W3C VC model.

ZK over existing credentials. zk-creds (Rosenberg et al., 2023) produces zk-SNARK proofs over identities whose authenticity is attested by an existing infrastructure, such as ICAO 9303 passports or printed government ID cards. As opposed to zk-creds, which positions itself outside the BBS+/W3C-VC stack, our work targets a BBS+ credential issued under the W3C VC Data Model, with the SNARK proving conditions over its hidden attributes rather than re-proving the issuer's signature. The survey of Flamini et al. (2024) provides a comprehensive overview of selective-disclosure mechanisms for VCs, distinguishing hiding-commitment schemes (such as SD-JWT and ISO mdoc) from zero-knowledge-proof-based schemes (such as CL and BBS); SNARK-based predicate hiding inside W3C presentations is covered only very sparsely.

Dock/Truvera. The closest deployed system, the dock.io project (Truvera) (Dock Labs, 2024), combining BBS+

with LegoGroth16 predicate proofs (including compiled Circom circuits executed on BLS12-381 via arkworks) is available both in Rust and as JavaScript/WASM packages, and can issue W3C Verifiable Credentials. The difference from PrivaCred is one of construction. Dock relies on LegoGroth16-specific gadgets and keeps its predicate proofs in a bespoke presentation format. We instantiate the predicate layer with a standard Groth16 circuit on BN254, proved and verified by mainstream snarkjs, bound to the credential by a Poseidon commitment inside a standard W3C Verifiable Presentation. LegoSNARK (Campanelli et al., 2019) is the modular SNARK framework that motivates "link-and-prove" designs. We draw the binding intuition from it but instantiate the binding with a Poseidon commitment inside a standard Groth16 circuit rather than with LegoGroth16-specific gadgets, to retain the mainstream circom/snarkjs toolchain. Recent ZKP benchmarks (Oude Roelink et al., 2024) support our choice of Groth16 over Bulletproofs on the size and verify-time axes. PrivaCred additionally supports PLONK as an alternative universal-setup proving system. This demonstrates that the BBS+/SNARK binding (Contribution 2) is independent of the underlying proof system.

Predicate-language tooling. General-purpose ZK-DSLs such as Circom (iden3, 2024a), Noir (Aztec Network, 2024), Leo and Cairo emit arbitrary R1CS or AIR programs. That breadth costs something. The audit surface is large, and whoever writes the circuit needs a great deal of training. At the other extreme, VC-specific predicate tooling such as Dock's composite-proof builder (Dock Labs, 2024) targets a fixed predicate family but binds it to a stack-specific proof system. The Predicate Compiler in this dissertation (Section IV-D) sits between the two. It is VC-specific, composes four constraint kinds with AND or OR, and emits Circom against a mainstream toolchain. It is small on purpose, so that one reader can audit all of it.

Research-gap statement. The deployed system closest to PrivaCred, Dock/Truvera, combines BBS+ with zk-SNARK predicate proofs (LegoGroth16 and Circom-on-BLS12-381 constructions) (Dock Labs, 2024). To the best of our knowledge, that combination has not been demonstrated as an open end-to-end implementation built on the mainstream circom/snarkjs Groth16 toolchain on BN254 inside standard W3C Verifiable Presentations; PrivaCred targets that construction gap, a toolchain and data-model integration gap rather than a new cryptographic primitive, instantiating the commit-and-prove paradigm (Campanelli et al., 2019) with a hash commitment as the cross-curve bridge. That gap has five components, each left open by prior work: the absence of a complete BBS+ W3C VC prototype, of an experimental study of cross-verifier correlation, of a SNARK-backed attribute-hiding mechanism inside a VC, of a clean binding mechanism between the BBS+ and SNARK curves, and of a system in which attribute values are never disclosed in cleartext.

IV. System Design

The contribution in plain terms. A verifier usually needs one narrow fact — is this holder over 18, is this holder a student — and current systems answer by handing over the whole record that contains it. PrivaCred answers the question instead: the verifier learns one yes-or-no bit plus the query parameters it supplied itself, and no attribute value at all. The cryptography is standard. What changed is where one value sits. BBS+ signatures are fixed to one elliptic curve while the practical proving toolchain runs on another, and a value from one curve is meaningless on the other. PrivaCred makes a single hash commitment both the only message the signature covers and an input the proof must account for, so the verifier joins the two layers by checking that one integer matches on both sides. It needs no new primitive and no bespoke linking gadget. The construction stays on mainstream tooling, so a reader can download it and re-run the result.

Requirements. The construction must: (R1) disclose no raw attribute value; (R2) support range, equality and set-membership predicates over the hidden attributes; (R3) bind the signature and predicate-proof layers so they cannot be mixed across credentials; (R4) verify offline; (R5) support revocation; (R6) resist cross-verifier correlation; and (R7) stay on a mainstream, auditable toolchain. Appendix F maps each to the contribution that meets it and the experiment that verifies it.

PrivaCred has three roles. The issuer samples a 254-bit blinding scalar, computes a Poseidon commitment over the holder's attributes, and signs the commitment with a BBS+ key under the BLS12-381 curve. The wallet stores the credential together with the blinding, derives a verifier-specific pairwise DID, generates a BBS+ selective-disclosure proof revealing only the commitment, and generates a zk-SNARK proof (Groth16 or PLONK) attesting that the underlying attributes satisfy the predicate. The verifier receives a single Verifiable Presentation, runs nine checks (schema, BBS+, commitment extraction, SNARK, binding, predicate parameters, validity bit, replay, revocation), and decides accept or reject. No party contacts the issuer at verification time (Figure 1).

A. Contribution 1: the Circom Circuit

The single novel circuit, PrivaCredFullProof, takes four private inputs (age, student, institutionHash, blinding) and four public inputs (commitment, ageThreshold, requiredStudent, requiredInstitution). It enforces four constraints: the Poseidon-4 hash of the private inputs equals the public commitment; age is greater than or equal to ageThreshold, encoded via an 8-bit GreaterEqThan comparator; student equals requiredStudent; and institutionHash equals requiredInstitution. The combination of all four constraints in one R1CS yields a single proof (Groth16 or PLONK) whose verification reduces to a small constant number of pairing checks plus a multi-scalar multiplication over the public inputs. Section IV-D

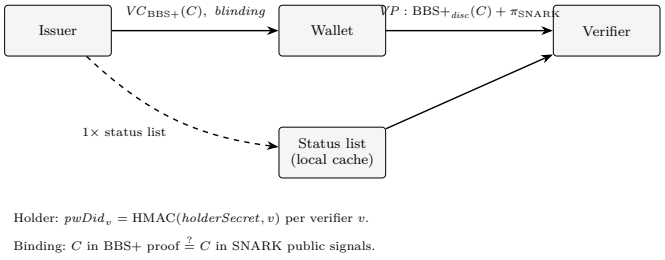


Fig. 1. PrivaCred architecture. Solid arrows are per-session messages; the dashed arrow from issuer to the status-list cache is one-time. The verifier never contacts the issuer at verification time. C is the Poseidon commitment over all attributes; the binding check ties the BBS+ layer (BLS12-381) and the SNARK layer (BN254) by hash equality, curve-independently.

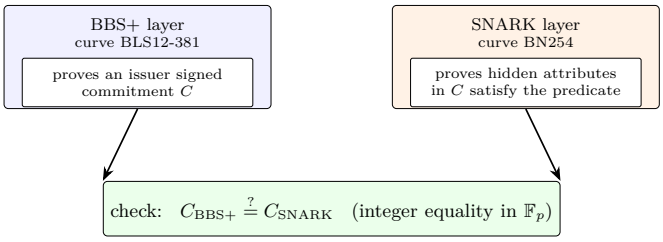


Fig. 2. Contribution 2: the curve-independent binding. BBS+ and the SNARK live on different curves; the verifier ties them together with a single field-element equality on the Poseidon commitment C.

below generalises this hand-written circuit to a Predicate Compiler.

B. Contribution 2: Curve-Independent Binding

BBS+ is mandated on the BLS12-381 curve by the W3C BBS suite (Bernstein and Sporny, 2026), while circomlib's Poseidon and the public Hermez Powers of Tau ceremony target the BN254 (alt_bn128) curve. A fully unified-curve construction would require porting Poseidon to BLS12-381 and either re-running a powers-of-tau ceremony or relying on an unaudited one, both prohibitive in an MSc time budget.

BBS+ signs the 32-byte big-endian encoding of $C = \mathrm{Poseidon}(\mathrm{age}, \mathrm{student}, \mathrm{instH}, \mathrm{blind})$ over the four private attribute inputs as a single message. C is published in the credential, and at presentation time the wallet uses a BBS+ selective disclosure to reveal message index 0, yielding $C_{\mathrm{BBS+}}$. Separately, from the public input C, the wallet computes the public signal C_{SNARK} of the SNARK circuit PrivaCredFullProof. Acceptance turns on one test: whether the two integers in $\mathbb{F}_p$ are equal, $C_{\mathrm{BBS+}} = C_{\mathrm{SNARK}}$ (Figure 2). Because this is a comparison of two integers in $\mathbb{F}_p$ for p the BN254 scalar order, the curves of the underlying proof systems are irrelevant to the binding. A single hash equality therefore carries the whole binding, instantiating the link-and-prove pattern (Campanelli et al., 2019) as a Poseidon hash equalised in a standard R1CS (for Groth16 here, but equally for any SNARK with a public-signal vector, e.g. PLONK). The trade-off is a ∼256-bit hash equality as the inter-layer trust anchor.

Proof-system independence. The binding equation $C_{\mathrm{BBS+}}$ = publicSignals[PS_COMMITMENT] does not reference the proof system at all. Groth16 (Groth, 2016) and PLONK consume the identical circuit and emit the same five-tuple, so proof-system independence is demonstrated here rather than argued.

C. Contribution 3: Presentation JSON Layout

The W3C VC Data Model does not specify where to place the predicate proofs of a presentation within a Verifiable Presentation. We place them in an array predicateProofs[] at the same level of the Verifiable Presentation as the proof of the Verifiable Credential that triggered it. The semantic difference is that the credential's proof is a signed assertion by the issuer, whereas each predicateProof is an assertion by the holder that, at presentation time, the hidden attributes satisfy the conditions specified in the PresentationRequest. Every element of predicateProofs[] relates to the presentation, whereas the credential's proof relates to the individual credential. Each element carries five fields: the protocol used to prove the predicate ("groth16" or "plonk"); the public-input commitment used by the SNARK; the proof object itself; the five-tuple of public inputs of the circuit; and a human-readable representation of the predicate for audit logs.

TABLE I
Worked example. A holder aged 21, a student at QMUL, answers the policy age ≥ 18 AND student $= 1$ AND institution $=$ QMUL. Values are from a real run; the presentation is 2,660 B on the wire, consistent with the n=100 mean in Table II.

Held by the wallet, never transmitted	age=21; student=1; institution=QMUL; 254-bit blinding
On the wire	pairwise holder DID (fresh per verifier); ageCommitment (Poseidon, hiding); BBS+ derived proof revealing message index 0; Groth16 proof on BN254; publicSignals =[1, C, 18, 1, H(QMUL)]; the predicate string
Verifier learns	one bit — that the hidden attributes satisfy the predicate — plus the query parameters it chose itself
Verifier cannot learn	that the holder is 21 rather than any other age ≥ 18; the student and institution values; the blinding

Table I makes the disclosure boundary concrete for a single presentation. The only holder-derived value on the wire is the hiding Poseidon commitment. The remaining public signals are the threshold, the student flag and the institution hash that the verifier supplied in its own request, echoed back so that both parties agree on what was proved. The verifier therefore learns that the holder is at least 18, and cannot distinguish a holder aged 21

from any other holder satisfying the same predicate. One string in the payload deserves comment: the institution name appears once, inside the human-readable predicate field, because it is the condition the verifier asked about rather than a disclosure of the holder's attribute.

D. Contribution 1 (extended): the Predicate Compiler

PrivaCredFullProof hard-codes a single predicate family. Any verifier wanting different predicates would otherwise need their own hand-written Circom file. Instead we developed a Predicate Compiler, a TypeScript module (packages/compiler/) that takes a Predicate value, a JSON object with an attributes field and four kinds of constraint joined by AND or in disjunctive normal form. From that it generates a single Circom source file for the predicate, together with a trusted-setup pipeline that runs the generated circuit against Hermez Powers of Tau. The outputs are registered in a JSON CircuitRegistry held by the wallet and each verifier, keyed by the predicate-hash of the Predicate. Each entry's values are the wasm, zkey and vkey generated for the circuit. The wallet's generatePresentationForCircuit and the verifier's compiler-path branch use the registry to find the correct wasm, zkey, vkey and public-input layout for a given circuit.

A Predicate value is a list of attribute specs (of type uint, bool or hash) joined by AND, together with four kinds of constraint. Range constraints on uint attributes use circomlib's GreaterEqThan and LessEqThan. Equality constraints on bool or hash attributes are straightforward === assertions. An in constraint tests whether an element lies in a small finite set, comparing it against every set element with IsEqual and folding the boolean results into a single indicator bit. A set-large constraint tests membership in a large set via Merkle-of-Poseidon inclusion of depth $d = \lceil \log_2 K \rceil$, with the element list condensed into one public root. The per-constraint validity bits are hard-pinned to 1 as defence in depth, and the compiler always injects a private blinding and a public commitment constrained by Poseidon over all attributes plus the blinding.

Our scope is deliberately limited: we do not aim to implement a full ZK-DSL like Circom (iden3, 2024a) or Noir (Aztec Network, 2024). The compiler emits only Circom and supports AND and OR composition over the four constraint kinds. For an OR composition it folds a disjunction over the AND-clauses. The per-gadget hard pins are dropped inside an OR (where they would force every clause to hold at once) and soundness is instead carried by the final valid === 1 pin. A SHA-256 hash of the Predicate JSON is the registry key, and for AND-only predicates that hash omits the clauses field so pre-OR-extension hashes stay identical. In total we support nine demo predicates across five families: student-at-qmul (the canonical one), two-sided ranges, linear sets up to 16 elements, OR-of-conjunctions, and Merkle-of-Poseidon set membership up to K=150.

The wallet gains a new entry point to generate a presentation for any compiled circuit, while generatePresentation() still targets PrivaCredFullProof. The verifier's parallel branch finds the descriptor in its local registry by predicate-hash and reads the vkey and public-input layout from local storage rather than the wire. The expected public-input values come from the verifier's own policy configuration, passed to snarkjs.{groth16,plonk}.verify.

E. Security Argument

A game-based sketch in Appendix A argues four properties under standard assumptions. Each proving system is treated in the idealised-group model in which it is analysed: the generic-group model for Groth16 (Groth, 2016), the algebraic-group model for PLONK (Gabizon et al., 2019). The argument relies on Poseidon's collision- and pre-image-resistance rather than modelling it as a random oracle: (P1) presentation unforgeability reduces to BBS+ EUF-CMA on BLS12-381 (Au et al., 2006; Tessaro and Zhu, 2023), Groth16/PLONK knowledge soundness on BN254 (Groth, 2016; Gabizon et al., 2019), and Poseidon collision resistance (Grassi et al., 2021); (P2) attribute hiding reduces to SNARK zero-knowledge, BBS+ derived-proof honest-verifier ZK, and computational hiding of Poseidon under the per-credential blinding; (P3) as explained in Appendix A-D, the prototype realises bounded N-show unlinkability for a commitment pool whose size is the number of independently BBS+-signed credentials the issuer generates for the same set of attributes. Because each credential in the pool has its own blinding, pseudonym and status index (E2), even colluding verifiers do not learn the identifiers of the presented member. The guarantee is limited to the number of mints the issuer declared for the pool. Proving unbounded N-show unlinkability would require a cross-curve opening-equality proof between BLS12-381 and BN254; (P4) binding soundness between the BBS+ and SNARK layers is a direct corollary of P1, since Step 5 of the verifier explicitly checks $C_{\text{BBS+}} \stackrel{?}{=} C_{\text{SNARK}}$. The concrete advantage bound at the prototype's $\lambda \approx 128$ parameters, and the scope limits of the argument (it covers the construction in idealised-group models, not the running TypeScript code), are given in Appendix A-F.

V. Implementation

PrivaCred is a TypeScript (5.4, strict mode) monorepo of 11 workspaces: circuits, core, issuer, wallet, verifier, the predicate compiler, three baselines, and a browser demo server and client. The implementation targets Node.js 20 LTS (benchmarks ran on Node v25.9.0). The cryptographic stack uses circom 2.0 for circuit compilation, snarkjs for Groth16 and PLONK proving and verification, circomlib/circomlibjs for Poseidon, and @digitalbazaar/bbs-signatures for the BBS+ suite. The evaluation code is written in Python 3.11 using NumPy,

pandas and matplotlib. Table VI enumerates the major dependencies.

BBS+ canonicalisation contract. BBS+ signs an array of opaque byte messages of arbitrary length. We fix a single-element message vector whose sole 32-byte entry is the big-endian encoding of the Poseidon commitment (see packages/wallet/src/prove.ts). This sidesteps URDNA2015 JSON-LD canonicalisation (a known source of subtle bugs in W3C VC implementations) because there is exactly one message and it is a fixed-width binary value. The trade-off is that the commitment is the only thing the holder can ever disclose. All other attribute information must flow through the SNARK predicate.

Public-signal wire format. The snarkjs convention serialises public signals as outputs first, then the declared public inputs in declaration order. For our circuit this gives the five-tuple [valid, commitment, ageThreshold, requiredStudent, requiredInstitution], where valid is the comparator output bit. An early design sketch, predating the snarkjs integration, showed only four fields. This surfaced as a wallet/verifier mismatch in packages/verifier/src/verify.ts during integration testing, which we then pinned by enforcing publicSignals[0] = "1" in the verifier pipeline as defence in depth.

Status-list caching. A core design requirement is that the issuer must not be contacted in the verification flow, both to preserve the privacy property and to keep verification offline. We cache a Bitstring Status List downloaded from the issuer in a persistent store on local disk (packages/verifier/src/revocation.ts) on every successful refresh. A verifier can additionally poll a configured rotation source for signed status lists, accepting a new list only when version, issuer and signature all check, so that downgrade, timestamp-rollback and issuer-substitution attempts are rejected. This implements a real-world signed-rotation pattern.

The verifier's nine-step pipeline (packages/verifier/src/verify.ts; Figure 12) aborts on the first failed step with a precise audit reason (schema mismatch, bbs+ proof invalid, snark proof invalid, binding mismatch, predicate mismatch, predicate not satisfied, or revoked). Step 5 is the binding check, the cryptographic heart of Contribution 2. For the prototype we deliberately sign the BBS+ message vector directly via core.sign rather than threading a JSON-LD documentLoader through @digitalbazaar/vc, a documented prototype simplification, not a production posture.

VI. Evaluation

A. Methodology

evaluation/analyse.py runs the experiments and analyses their output, so the whole evaluation reproduces from a single command. Of the nine experiments, E1–E7 are driven from that script, while E8 (the compiler experiment) and E9 (the concurrency experiment) are run with test harnesses written for the purpose. E3 and E7 run twice (once per proving system), E3 supporting the side-by-side comparison in Table II. The other experiments are protocol-agnostic and run once. To get a fair sample for E2 (linkability) and E3 (performance) we follow the 50 holders × 10 verifiers × 5 sessions design with a seeded NumPy random number generator (seed 42). Performance is the mean of $n = 100$ iterations, measured for Node code using performance.now() and for Python code using time.perf_counter(). Performance of the PrivaCred implementation is measured by running an in-process Node benchmark for generatePresentation() and verifyPresentation() on a newly issued credential. The baseline figures come from the mock servers in packages/baselines/. The three baselines are functional stand-ins rather than like-for-like counterparts: each performs a signature or token check and no zero-knowledge work at all, so the latency column of Table II should be read as the cost of the privacy property rather than as a competitive benchmark. Their role in this evaluation is the disclosure and linkability comparison (E1, E2, E7), where the wire shapes are directly comparable. Without a trusted setup for a given circuit, the test harness reports skipped: circuit not compiled; it does not make up any results.

B. Results

Four findings carry the contributions, one for each axis named in (O3). PrivaCred disclosed zero holder attribute values where the baselines disclosed one to seven (E1, E7). Two colluding verifiers correlated 0% of PrivaCred holders against 100% for every baseline, at the protocol level and bounded by the pool size (E2). Verification completed offline in 47–49 ms under both proving systems (E3). The system held under the robustness checks, rejecting every revoked credential, every replay and every predicate the boundary grid said it should reject (E4–E6). The nine experiments below report the full results (headline figures in Tables II, VII and Appendix B).

E1 Disclosure. For a single prove-student-over-18 flow the disclosed-value counts are PrivaCred 0, KYC 7, OAuth 2 (sub, email) and SIWE 1 (wallet address). The OAuth figure is from an HS256 mock IdP. A real Google path discloses a strict superset, typically eight claims, which widens rather than narrows the gap. Figure 4 (Appendix C) plots the comparison.

E2 Linkability under collusion. Two colluding verifiers try to match a holder's sessions across their logs by identifier, by value and by session timing. Every baseline links 100% of holders on a stable identifier (documentNumber, sub, wallet address) and again on cleartext values; PrivaCred links 0% on both, because each verifier sees a different pairwise DID and no attribute value is disclosed. The by-value result assumes a fresh commitment per session, which the N-show pool supplies and its exhaustion removes (Section VII). Timing yielded no linkage for any system, which is an artefact of the protocol-level simulation rather than a property, and is revisited under

TABLE II
E3, side by side: PrivaCred under Groth16 vs PLONK vs baselines. The baselines do no zero-knowledge work, so the latency gap is the cost of the privacy property, not a competitive result ($n = 100$ iterations, mean latency and mean payload size; numbers populated from evaluation/results/e3_performance.summary.json).

System	Prove (ms)	Verify (ms)	Payload (B)
PrivaCred-Groth16	96.4	46.57	2,661
PrivaCred-PLONK	2,248.1	48.70	4,035
KYC	0.55	0.40	258
OAuth	0.77	0.36	206
SIWE	0.38	0.45	204

threats to validity (Section VII). Figure 5 (Appendix C) summarises the outcome.

E3 Performance. Table II reports the full prove–verify round-trip on the benchmark machine (Apple M4, 16 GB, Node v25.9.0). The result that the table does not show is where the time goes: verification latency under both proving systems is dominated by the BBS+ derived-proof check, not by the SNARK verify, so the constant-time SNARK is not the cost centre a reader would expect. Figure 6 (Appendix C) plots prove-side latency on a log scale.

E4 Revocation. Whether a credential has been revoked must be answerable offline. The status-list index for each revoked credential was flipped to 1. Across 10 credentials, five valid and five revoked, the verifier accepted every valid one and rejected every revoked one with no issuer contact at verify time (evaluation/results/e4_revocation.csv; Figure 7, Appendix C). The rotation-attack cases behind this result are in packages/verifier/test/revocation.test.ts.

E5 Replay. A verifier-issued nonce is bound into the BBS+ presentation header. A presentation is accepted only on its first submission. Replays are rejected because each fresh verifier issues a fresh nonce. Across 50 sessions every first submission was accepted and every replay rejected (evaluation/results/e5_replay.csv; Figure 8, Appendix C).

E6 Predicate correctness. On a test grid of predicate boundary cases encoded by an independent reference implementation, the compiled circuit agreed with the reference on every case for each named predicate, which is what confirms the circuit encodes the intended predicate rather than one that merely passes the demo inputs (evaluation/results/e6_predicate.csv; Figure 9, Appendix C).

E7 Payload audit. A regular-expression scan of every wire payload for cleartext attribute values returns zero adjacent-to-key matches in PrivaCred (e.g. no "age":21 pattern in the JSON), while KYC and OAuth each expose the holder's email as a cleartext attribute string. Engaging the real Google OAuth path widens this gap with cleartext name, picture and locale fields alongside sub and email. We report a stronger adjacent-to-key scan and a weaker raw-substring scan. Under the weaker scan PrivaCred shows two matches, both benign: "21" occurring by chance inside the base64 proofValue, and "QMUL" in the predicate field, which records the verifier's own query rather than a holder attribute. No holder attribute value is recoverable under either scan. SIWE returns zero under both, which understates its disclosure rather than describing a privacy property, since the wallet address it does disclose is counted in E1 and drives its 100% linkage in E2. Both scans are in e7_payload_audit.csv; Figure 10 (Appendix C) compares them.

E8 Predicate Compiler. The Predicate Compiler is tested using nine demo predicates (Section IV-D). Six are reported in Table VII (Appendix D). The full nine-demo data is in evaluation/results/e8.csv, visualised in Figure 11. For each demo we measure the R1CS constraint count, the Groth16 and PLONK prove and verify latencies (means of three iterations on the E3 hardware), and the wire-format proof byte-size in the snarkjs JSON encoding.

The compiler-emitted student-at-qmul circuit has 753 R1CS constraints versus 307 for the hand-written PrivaCredFullProof. The overhead comes from redundant pins the compiler emits deliberately: the per-constraint validity signals and their AND-fold chain, and the defence-in-depth hard pins at each gadget output (e.g. a*(1-a)===0 per bool attribute), because the emitted code runs unaudited in other verifiers' wallets. Second, verify times under both proving systems are essentially flat across the demos, between 5.1 and 7.6 ms, since the verifiers are linear in the public-input count rather than the constraint count. Prove times under PLONK are 4.8–11× slower than under Groth16 at these sizes, consistent with the universal-setup overheads reported for PLONK at small circuit sizes (Gabizon et al., 2019). Third, the OR-fold is near-free at this scale: eu-or-uk-citizen is within 1.1% of the AND-only member-of-three-gyms, and student-at-qmul-or-staff-at-qmul within 1% of student-at-qmul: adding the second clause costs just four extra constraints (753→757). Fourth, set-large (Merkle-of-Poseidon) beats linear probes only above $K \approx 64$: at K=27 it costs 3,132 constraints versus ∼1,100 linear, at K=150 4,787 versus ∼5,000.

E9 Concurrency. Reusing the canonical credential, the end-to-end prove+verify pipeline was run under closed-loop bounded concurrency $C \in \{1, 2, 4, 8\}$ (60 timed iterations per C for Groth16 and 20 for the PLONK reference run, with 5 warm-up iterations discarded per the JMH warm-up-then-timed methodology (Shipilëv and OpenJDK code-tools team, 2014); throughput/latency plot in Figure 3, Appendix B). Throughput plateaus at around 8 presentations/s while P50 latency grows roughly linearly in C (140, 250, 513, 907 ms), which we attribute to single-threaded snarkjs WASM witness generation saturating one core. P99 stays within 1.2× of P50. PLONK is measured only at C=1 (∼2.3 s/proof). Closed-loop measurement under-reports tail latency under overload (coordinated omission; recorded in the summary JSON).

VII. Discussion

Limitations. The JSON-LD context for our predicateProofs[] extension is defined and bundled with the supporting material as contexts/privacred-v1.jsonld. Its public URL (w3id.org/privacred/v1, administered by the W3C Permanent Identifier Community Group) has its redirect registration pending, and strict JSON-LD processors verify against the bundled file until it lands. Moving both layers onto one curve is not practical. BBS+ and the SNARK each stay on the toolchain that is mature for it.

Trusted setup. The trusted setup for verification of a single credential is the public multi-party Hermez Powers of Tau ceremony (54+ contributions, documented in the snarkjs registry (iden3, 2024b)). Groth16 then runs a single Phase 2 for the newly created circuit; PLONK derives its verification keys deterministically from the same ptau. All reproducibility lives in packages/circuits/scripts/setup.sh. Revocation reuses the signed StatusListCache and rotation pipeline of Section V, with an optional strict mode that fails closed when the cache is empty or older than the configured maxAgeMs. The remaining production-deployment work is gossip between verifiers. The wallet ships as a single-user CLI. Multi-credential support, key recovery, and binding of credentials to a hardware secure element are out of scope for this prototype.

Is PLONK's overhead an artefact or structural? Constraint counts are not one-to-one (R1CS omits the permutation bookkeeping PLONK carries), so the direct comparison is prove time. Our experiments put that ratio at roughly 23× on the canonical circuit, and it grows rather than amortising away with circuit size: from 4.8–6.0× on the sub-800-constraint AND-only predicates to 8.9× at 3,132 and 11.2× at 4,787 constraints, most pronounced for the Poseidon-heavy Merkle circuits (Gabizon et al., 2019); Table III sets the same comparison out as a deployment choice. This appears structural to the tooling for this proof system. The overhead matters most where the Predicate Compiler lives: every compiler-emitted circuit needs its own Groth16 Phase-2 ceremony (a separate setup-trust point per verifier-specific predicate), whereas PLONK derives its keys deterministically from the shared ptau. A deployment compiling many verifier-specific predicates may therefore rationally accept the slower prover to avoid per-predicate setup trust.

Where the system breaks beyond the flagged correlation channels. (i) Issuer-key compromise is silent and retroactive: BBS+ signs one opaque commitment, so a compromised issuer key can sign arbitrary attributes for a new credential with no forensic trace distinguishing forgery from legitimate issuance, and the prototype's credential revocation does not extend to revoking the issuer itself. (ii) In-browser proving is its own fingerprinting surface: the demo-web proving-WASM's timing and memory profile would be measurable by the hosting page, a property of this deployment choice, not the protocol. (iii) Coercion defeats selective disclosure: a verifier with enough leverage can simply demand the holder's attributes and blinding. The protocol has no duress mechanism, a human-layer problem no E1–E9 experiment can capture. (iv) Pool depletion is a silent trust cliff: exhausting the N-show pool degrades the holder to full linkability with no user-visible signal, and refill requests to the issuer would form a timing side channel that E2 does not capture.

TABLE III
Groth16 versus PLONK as a deployment choice. Verification cost, where a verifier's economics sit, is effectively identical; the entire penalty is prover-side and payload-side, and is paid against a per-predicate setup-trust saving.

	Groth16	PLONK
Prove, canonical circuit (ms)	96.4	2,248.1
Prove, nine compiled predicates	baseline	4.8–11× slower
Verify (ms)	46.57	48.70
Payload (B)	2,661	4,035
Throughput at C=1 (s^{-1})	7.1	0.44
Per-predicate trusted setup	Phase-2 ceremony	none (universal ptau)

Threats to validity. All experiments were performed at the protocol level. Real-world correlation also operates over network metadata, IP fingerprinting and TLS timing, so we under-report real-world linkability across all four systems. Performance numbers reflect a single hardware configuration. E1's fixed wire-shape disclosure counts are confirmed by the baselines' unit tests.

Legal, social, ethical and sustainability considerations. PrivaCred is a data-minimisation technology: it operationalises the GDPR minimisation principle (Art. 5(1)(c)) and aligns with the selective-disclosure requirements of eIDAS 2.0 (ETSI, 2025). The social benefit is a reduction in cross-service correlation and the de-anonymisation it enables (Narayanan and Shmatikov, 2008). The ethical risk runs the other way, since proof-of-eligibility infrastructure can exclude people who cannot obtain digital credentials. An auditable open-source implementation is therefore an ethical requirement. No human-participant data was collected, so no ethics approval was required. On sustainability, verification is offline at 47–49 ms of commodity-CPU work with no ledger anchoring. Proving cost (especially PLONK's ∼2.25 s) is the dominant energy factor and one reason Groth16 is the default path.

VIII. Conclusion and Future Work

PrivaCred is an end-to-end W3C Verifiable Credential system that combines BBS+ selective disclosure with full attribute-hiding zk-SNARK predicate proofs on the mainstream circom/snarkjs toolchain. Its three contributions close the identified gap: the single-circuit predicate conjunction, the curve-independent Poseidon binding, and the presentation-level proof layout. Across nine experiments PrivaCred disclosed no holder attribute values, and it admitted no identifier- or value-based correlation across colluding verifiers on the channels measured, bounded

to the pool size. Verification ran offline in 47–49 ms on the hardware of Table II. Correlation was evaluated at the protocol level only. The network-metadata channels named in Section VII remain out of scope.

Future work. Next steps: registering the JSON-LD @context; gossiped status-list rotations; EUDI/SD-JWT alignment under OpenID4VP; and unbounded unlinkability via a cross-curve opening-equality proof.

References

Alpár, G., van den Broek, F., Hampiholi, B., Jacobs, B., Lueks, W. and Ringers, S. (2017), IRMA: Practical, decentralized and privacy-friendly identity management using smartphones, in '10th Workshop on Hot Topics in Privacy Enhancing Technologies (HotPETs)'. Workshop programme: https://petsymposium.org/2017/hotpets.php.

Au, M. H., Susilo, W. and Mu, Y. (2006), Constant-size dynamic k-TAA, in 'Security and Cryptography for Networks (SCN)', Vol. 4116 of Lecture Notes in Computer Science, Springer, pp. 111–125. Introduces the BBS+ variant of the Boneh–Boyen–Shacham group signature; doi: https://doi.org/10.1007/11832072_8.

Aztec Network (2024), 'Noir: a domain-specific language for SNARK proving systems', Source repository, https://github.com/noir-lang/noir.

Bellare, M. (2006), New proofs for NMAC and HMAC: Security without collision-resistance, in 'Advances in Cryptology — CRYPTO 2006', Vol. 4117 of Lecture Notes in Computer Science, Springer, pp. 602–619. doi: https://doi.org/10.1007/11818175_36.

Bernstein, G. and Sporny, M. (2026), 'Data integrity BBS cryptosuites v1.0', W3C Candidate Recommendation Draft, April 2026, https://www.w3.org/TR/vc-di-bbs/.

Boneh, D., Boyen, X. and Shacham, H. (2004), Short group signatures, in 'Advances in Cryptology — CRYPTO 2004', Vol. 3152 of Lecture Notes in Computer Science, Springer, pp. 41–55. doi: https://doi.org/10.1007/978-3-540-28628-8_3.

Camenisch, J. and Herreweghen, E. V. (2002), Design and implementation of the Idemix anonymous credential system, in 'Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)', ACM, pp. 21–30. doi: https://doi.org/10.1145/586110.586114.

Camenisch, J. and Lysyanskaya, A. (2001), An efficient system for non-transferable anonymous credentials with optional anonymity revocation, in 'Advances in Cryptology — EUROCRYPT 2001', Vol. 2045 of Lecture Notes in Computer Science, Springer, pp. 93–118. doi: https://doi.org/10.1007/3-540-44987-6_7.

Campanelli, M., Fiore, D. and Querol, A. (2019), LegoSNARK: Modular design and composition of succinct zero-knowledge proofs, in 'Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)', ACM, pp. 2075–2092. doi: https://doi.org/10.1145/3319535.3339820.

Chang, W., Rocco, G., Millegan, B., Johnson, N. and Terbu, O. (2021), 'EIP-4361: Sign-in with Ethereum', Ethereum Improvement Proposal, https://eips.ethereum.org/EIPS/eip-4361.

Dock Labs (2024), 'Composite proof system with BBS+ signatures, LegoGroth16 and Circom R1CS circuits', Open-source library, https://github.com/docknetwork/crypto; Circom integration announcement, https://blog.dock.io/circom-language-integration/.

ETSI (2025), Electronic signatures and trust infrastructures (ESI); analysis of selective disclosure and zero-knowledge proofs applied to electronic attestation of attributes, Technical Report TR 119 476, V1.3.1, European Telecommunications Standards Institute. Available at: https://www.etsi.org/deliver/etsi_tr/119400_119499/11947601/01.03.01_60/tr_11947601v010301p.pdf (Accessed: 10 July 2026).

Fett, D., Küsters, R. and Schmitz, G. (2016), A comprehensive formal security analysis of OAuth 2.0, in 'Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)', ACM, pp. 1204–1215. doi: https://doi.org/10.1145/2976749.2978385.

Fett, D., Yasuda, K. and Campbell, B. (2025), 'Selective disclosure for JWTs (SD-JWT)', IETF OAuth Internet-Draft, draft-ietf-oauth-selective-disclosure-jwt-22, May 2025, https://datatracker.ietf.org/doc/draft-ietf-oauth-selective-disclosure-jwt/.

Flamini, A., Sciarretta, G., Scuro, M., Sharif, A., Tomasi, A. and Ranise, S. (2024), 'On cryptographic mechanisms for the selective disclosure of verifiable credentials', Journal of Information Security and Applications (JISA) 83, 103789. doi: https://doi.org/10.1016/j.jisa.2024.103789.

Frigo, M. and abhi shelat (2026), 'Anonymous credentials from ECDSA', IACR Communications in Cryptology 3(1). Article 7, https://cic.iacr.org/p/3/1/7. DOI: 10.62056/a3qjmpgxq. Preprint: IACR Cryptology ePrint Archive, Report 2024/2010.

Gabizon, A., Williamson, Z. J. and Ciobotaru, O. (2019), 'PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge', IACR Cryptology ePrint Archive, Report 2019/953, https://eprint.iacr.org/2019/953.

Goldwasser, S., Micali, S. and Rackoff, C. (1989), 'The knowledge complexity of interactive proof systems', SIAM Journal on Computing 18(1), 186–208. doi: https://doi.org/10.1137/0218012.

Grassi, L., Khovratovich, D., Rechberger, C., Roy, A. and Schofnegger, M. (2021), Poseidon: A new hash function for zero-knowledge proof systems, in '30th USENIX Security Symposium (USENIX Security 21)', USENIX Association, pp. 519–535. Available at: https://www.usenix.org/conference/usenixsecurity21/presentation/grassi (Accessed: 10 July 2026).

Groth, J. (2016), On the size of pairing-based non-interactive arguments, in 'Advances in Cryptology — EUROCRYPT 2016', Vol. 9666 of Lecture Notes in Computer Science, Springer, pp. 305–326. doi: https://doi.org/10.1007/978-3-662-49896-5_11.

iden3 (2024a), 'Circom: a circuit compiler for zkSNARKs', Source repository, https://github.com/iden3/circom.

iden3 (2024b), 'snarkjs: zkSNARK implementation in JavaScript and WASM', Source repository and Powers-of-Tau ceremony registry, https://github.com/iden3/snarkjs.

Looker, T., Kalos, V., Whitehead, A. and Lodder, M. (2026), 'The BBS signature scheme', IRTF CFRG Internet-Draft, draft-irtf-cfrg-bbs-signatures-10, January 2026, https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/.

Narayanan, A. and Shmatikov, V. (2008), Robust de-anonymization of large sparse datasets, in '2008 IEEE Symposium on Security and Privacy (S&P)', IEEE, pp. 111–125. doi: https://doi.org/10.1109/SP.2008.33.

Oude Roelink, B., El-Hajj, M. and Sarmah, D. (2024), 'Evaluating the efficiency of zk-SNARK, zk-STARK, and Bulletproof in real-world scenarios: A benchmark study', Information 15(8), 463. doi: https://doi.org/10.3390/info15080463.

Paquin, C., Policharla, G.-V. and Zaverucha, G. (2024), 'Crescent: Stronger privacy for existing credentials', IACR Cryptology ePrint Archive, Report 2024/2013, https://eprint.iacr.org/2024/2013.

Pfitzmann, A. and Hansen, M. (2010), A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management, Technical report, TU Dresden. Version 0.34. Available at: https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf (Accessed: 10 July 2026).

Rosenberg, M., White, J., Garman, C. and Miers, I. (2023), zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure, in '2023 IEEE Symposium on Security and Privacy (S&P)', IEEE, pp. 790–808. doi: https://doi.org/10.1109/SP46215.2023.10179430.

Sakimura, N., Bradley, J., Jones, M. B., de Medeiros, B. and Mortimore, C. (2014), 'OpenID Connect Core 1.0', OpenID Foundation specification, https://openid.net/specs/openid-connect-core-1_0.html.

Shipilëv, A. and OpenJDK code-tools team (2014), 'JMH: Java microbenchmark harness', OpenJDK project, https://openjdk.org/projects/code-tools/jmh/.

Sonnino, A., Al-Bassam, M., Bano, S., Meiklejohn, S. and Danezis, G. (2019), Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers, in '26th Annual Network and Distributed System Security Symposium (NDSS)', Internet Society. doi: https://doi.org/10.14722/ndss.2019.23272.

Sporny, M., Guy, A., Sabadello, M. and Reed, D. (2022), 'Decentralized identifiers (DIDs) v1.0', W3C Recommendation, 19 July 2022, https://www.w3.org/TR/did-core/.

Sporny, M., Longley, D., Prorock, M. and Alkhraishi, M. (2025), 'Bitstring status list v1.0', W3C Recommendation, 15 May 2025, https://www.w3.org/TR/vc-bitstring-status-list/.

Sporny, M., Thibodeau Jr., T., Herman, I., Cohen, G. and Jones, M. B. (2025), 'Verifiable credentials data model v2.0', W3C Recommendation, 15 May 2025, https://www.w3.org/TR/vc-data-model-2.0/.

Tessaro, S. and Zhu, C. (2023), Revisiting BBS signatures, in 'Advances in Cryptology — EUROCRYPT 2023', Lecture Notes in Computer Science, Springer, pp. 691–721. doi: https://doi.org/10.1007/978-3-031-30589-4_24.

Appendix A
Security Argument (extended)

This appendix expands the four-property game-based sketch summarised in Section IV-E. Following the model of Camenisch and Lysyanskaya (Camenisch and Lysyanskaya, 2001), each property is stated as a game between a challenger $\mathcal{C}$ and a probabilistic polynomial-time adversary $\mathcal{A}$; a reduction names the underlying assumption with its citation, and a concrete advantage bound is given at security parameter $\lambda = 128$. The argument is for the abstract construction in the idealised-group model appropriate to each proving system — the generic-group model for Groth16, the algebraic-group model for PLONK; it relies on Poseidon's collision- and pre-image-resistance at the circomlib parameters rather than modelling Poseidon as a random oracle, and it does not cover the running TypeScript code.

A. Notation

Table IV fixes notation. The BBS+ layer runs on BLS12-381 with scalar field of order $q \approx 2^{255}$ and pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. The SNARK layer runs on BN254 with scalar field $\mathbb{F}_p$ of order $r \approx 2^{254}$, which is the same field the Poseidon hash $H_{\text{Pos}} : \mathbb{F}_p^k \to \mathbb{F}_p$ operates over (parameters matching circomlib's poseidon gadget: $k = 4$ inputs, 128-bit collision target). The Poseidon commitment is $C = H_{\text{Pos}}(\text{age}, \text{student}, \text{instH}, \text{blinding})$ and is signed by BBS+ as a one-element message vector containing the 32-byte big-endian encoding of C. This encoding is injective and is compared as an integer, so the commitment equality the verifier checks is well-defined across the BLS12-381 and BN254 scalar fields.

B. P1: Presentation Unforgeability

Game $\mathsf{Game}_{\text{UF-VP}}$. Challenger generates $(sk_{\text{BBS}}, pk_{\text{BBS}}) \leftarrow \text{BBS.KeyGen}(1^\lambda)$ and a SNARK crs for the relation $\mathcal{R}_{\text{PrivaCred}}$, and hands $(pk_{\text{BBS}}, \text{crs})$ to $\mathcal{A}$. The adversary may query an issuance oracle on attribute tuples $(\text{age}, \text{student}, \text{instH})$ of its choice; the challenger

TABLE IV
Notation used in Appendix A.

Symbol	Meaning
λ	security parameter ($\lambda = 128$)
$sk_{\mathrm{BBS}}, pk_{\mathrm{BBS}}$	BBS+ issuer signing / verification key
crs	SNARK common reference string
H_{Pos}	Poseidon hash, $\mathbb{F}_p^4 \to \mathbb{F}_p$
C	Poseidon commitment over four attributes
σ	BBS+ signature over $[C]_{32\mathrm{B}}$
π_{bbs}	BBS+ derived (selective-disclosure) proof
π_{snark}	Groth16 or PLONK proof
n	verifier-issued nonce
$pwDid_v$	HMAC(holderSecret, v)
$\mathsf{Adv}^X_{\mathcal{A}}(\lambda)$	advantage of $\mathcal{A}$ against game X
$\mathsf{negl}(\lambda)$	negligible in λ

samples blinding $\leftarrow \mathbb{F}_p$, computes $C = H_{\mathrm{Pos}}(\cdot)$ and $\sigma = \mathrm{BBS.Sign}(sk_{\mathrm{BBS}}, [C])$, and returns $(\sigma, C, \mathrm{blinding})$. Eventually $\mathcal{A}$ outputs a presentation VP^* for a predicate $pred^*$. $\mathcal{A}$ wins iff (a) VP^* passes the verifier's nine-step pipeline (packages/verifier/src/verify.ts) and (b) no oracle query corresponded to attributes satisfying $pred^*$.

Reduction (three-case hybrid). We argue in three hybrids. Hybrid 0 is $\mathsf{Game}_{\text{UF-VP}}$ itself. Hybrid 1 replaces the honest crs with the simulator-extractor pair from the SNARK's knowledge soundness statement (Groth16 (Groth, 2016) Theorem 1; PLONK (Gabizon et al., 2019) §8; Groth16's statement is in the generic-group model, PLONK's in the algebraic-group model). The two hybrids are computationally indistinguishable in the corresponding model, with gap $\mathsf{negl}_{\mathrm{snark}}(\lambda)$. Hybrid 2 runs the extractor on $\mathcal{A}$'s winning proof to recover a witness $w^* = (\mathrm{age}^*, \mathrm{student}^*, \mathrm{instH}^*, \mathrm{blinding}^*)$ such that $H_{\mathrm{Pos}}(w^*) = C^*$ and $pred^*$ is satisfied. We case-split on C^*.

Case A: $C^* = C_i$ for some issuance query i whose attributes satisfied $pred^*$. This contradicts winning condition (b). $\Pr[\text{Case A}] = 0$.

Case B: $C^* = C_i$ for some query i whose attributes did not satisfy $pred^*$. Then $w^* \neq w_i$ but $H_{\mathrm{Pos}}(w^*) = H_{\mathrm{Pos}}(w_i)$, i.e. (w^*, w_i) is a Poseidon collision. This reduces to the collision resistance of Poseidon at the circomlib parameters (Grassi et al., 2021) with advantage $\mathsf{Adv}^{\mathrm{coll}}_{H_{\mathrm{Pos}}}(\mathcal{B}_1)$.

Case C: $C^* \notin \{C_i\}$. Then σ^* inside π^*_{bbs} is a valid BBS+ signature on a message $\mathcal{A}$ never queried, i.e. an EUF-CMA forgery. Reduces to BBS+ (Au et al., 2006; Tessaro and Zhu, 2023) (or equivalently the security considerations of the IETF draft (Looker et al., 2026)) with advantage $\mathsf{Adv}^{\text{EUF-CMA}}_{\text{BBS+}}(\mathcal{B}_2)$.

By the union bound,

$$\mathsf{Adv}^{\text{UF-VP}}_{\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\text{EUF-CMA}}_{\text{BBS+}}(\mathcal{B}_2) + \mathsf{Adv}^{\mathrm{coll}}_{H_{\mathrm{Pos}}}(\mathcal{B}_1) + \mathsf{negl}_{\mathrm{snark}}(\lambda).$$

At $\lambda = 128$ each term is at most $\approx 2^{-128}$, giving the prototype an aggregate advantage $\lesssim 2^{-126}$, which we report as "128-bit security by the standard margin convention".

C. P2: Attribute Hiding (Zero-Knowledge)

Game $\mathsf{Game}_{\mathrm{AH}}$. A real-versus-simulated game in the style of Goldwasser, Micali and Rackoff (Goldwasser et al., 1989). In World REAL the challenger produces an honest presentation from $(\mathrm{attrs}_0, \mathrm{blinding}, \sigma, pred)$; in World SIM a simulator produces a presentation from $pred$ and the public signals only. $\mathcal{A}$'s advantage is $|\Pr[\mathcal{A} = 1 \mid \mathrm{REAL}] - \Pr[\mathcal{A} = 1 \mid \mathrm{SIM}]|$.

Reduction (three-hybrid). (i) Replace the SNARK proof with the SNARK simulator (Groth16 ZK, Theorem 1 of (Groth, 2016); PLONK §8 of (Gabizon et al., 2019)); gap $\mathsf{Adv}^{\mathrm{ZK}}_{\mathrm{snark}}$. (ii) Replace the BBS+ derived proof with the derive-proof simulator (Au et al., 2006; Looker et al., 2026), which is honest-verifier zero-knowledge against pk_{BBS}; gap $\mathsf{Adv}^{\mathrm{ZK}}_{\text{BBS+}}$. (iii) Replace C with C' formed under a fresh uniform 254-bit blinding. Because the blinding is high-entropy and independent of the attributes, $C = H_{\mathrm{Pos}}(\mathrm{attrs}, \mathrm{blinding})$ is computationally hiding under Poseidon's pre-image resistance at the circomlib parameters (Grassi et al., 2021); we deliberately claim only computational hiding, not perfect hiding, because Poseidon is an arithmetic permutation rather than a Pedersen-style commitment, and we do not model Poseidon as a random oracle for this step.

$$\mathsf{Adv}^{\mathrm{AH}}_{\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathrm{ZK}}_{\mathrm{snark}} + \mathsf{Adv}^{\mathrm{ZK}}_{\text{BBS+}} + \mathsf{Adv}^{\mathrm{hide}}_{H_{\mathrm{Pos}}}.$$

D. P3: Bounded N-Show Unlinkability (commitment pool)

The gap we identified. The wallet's generatePresentation(·) (and the compiler-driven generatePresentationForCircuit(·)) read a single blinding from the local credential store, so the disclosed Poseidon commitment C, carried both in predicateProof.commitment and as publicSignals[1], is stable for a credential's lifetime. The disclosed verifiable credential also exposes three further stable identifiers: credentialSubject.id (the holder DID), credentialStatus.statusListIndex, and issuanceDate. Two colluding verifiers presented with the same credential could link the sessions by string equality on any of these.

The fix we implemented. We close the gap with a commitment pool that gives bounded N-show unlinkability. issueCredentialPool(·,N) (packages/issuer/src/issue.ts) mints N independently BBS+-signed credentials over the same attribute values, freshening every stable per-presentation identifier for each member: a fresh blinding (hence a distinct C); a fresh holder pseudonym (a proper did:key:z… derived from random key material); a distinct statusListIndex (the allocated contiguous range, revoked together); and an identical day-coarse issuanceDate, so the date does not distinguish one

holder's shows (fine-grained issuance timing remains a residual side channel, out of protocol scope, consistent with the network-metadata caveat of Section VII). The wallet (generatePresentationFromPool(·), packages/wallet/src/prove.ts) hands out one unspent member per presentation and marks it spent only after a successful proof. Two colluding verifiers therefore receive presentations sharing no disclosed identifier and cannot link them on the protocol-level channels analysed here. The guarantee is bounded twice over: to the pool size N (after N shows the holder obtains a fresh pool), and to the message content itself — network- and transport-layer metadata are out of scope (Section VII). The construction is exercised end-to-end by packages/wallet/test/pool.unlinkability.test.ts, which asserts that two presentations drawn from one pool carry distinct commitments, holder ids and status indices yet both verify (P4 binding intact).

What remains for unbounded unlinkability. Removing the bound would require the wallet to present a freshly re-randomised commitment on every show while proving, in zero knowledge, that it opens to the same BBS+-signed value: an opening-equality proof bridging the BLS12-381 BBS+ layer and the BN254 SNARK. The binding of Contribution 2 is deliberately realised by disclosing C in the clear, to avoid that cross-curve equality argument; unbounded unlinkability and the curve-independent binding are therefore in tension, and we leave the cross-curve construction to future work (Section VIII).

Reconciliation with experiment E2. E2 (Section VI; evaluation/experiments/e2_linkability.py) reports 0% linkability for PrivaCred across two colluding verifiers under a session envelope that sets value_signature = None on the assumption that the commitment is re-randomised per session. The pool brings the implementation into line with that idealisation for up to N shows: with pooled presentations there is no stable disclosed identifier, so the 0% colluding-verifier result now holds for the running code, not only the simulation, bounded to the pool size. In single-credential mode (which we retain) the original caveat stands and the commitment links; we document the distinction rather than overclaim. The empirical gap to the baselines (100% on OAuth-by-sub, SIWE-by-address, KYC-by-document) is unchanged.

Reduction (within the pool budget). For up to N shows (one fresh member per session) the claim is the standard one: pairwise DIDs are pseudorandom under HMAC-as-PRF (Bellare, 2006), BBS+ derived proofs are unlinkable by per-presentation randomisation (Au et al., 2006; Looker et al., 2026), and the SNARK proof carries fresh randomness on every prove call. The combined advantage of a distinguisher is bounded by

$$\mathsf{Adv}^{\mathrm{LINK}}_{\mathcal{A}}(\lambda) \le 2\mathsf{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}} + 2\mathsf{Adv}^{\mathrm{unlink}}_{\mathrm{BBS+}} + 2\mathsf{Adv}^{\mathrm{ZK}}_{\mathrm{snark}}.$$

E. P4: Binding Soundness

Game $\mathsf{Game}_{\mathrm{BIND}}$. $\mathcal{A}$ outputs a presentation VP that passes all verifier steps and for which the SNARK extractor returns $w^* = (\ldots, \mathrm{blinding}^*)$ with $H_{\mathrm{Pos}}(w^*) = C^*_{\mathrm{SNARK}}$; let $C^*_{\mathrm{BBS+}}$ be the commitment carried in π^*_{bbs}. $\mathcal{A}$ wins iff VP verifies and $C^*_{\mathrm{SNARK}} \neq C^*_{\mathrm{BBS+}}$.

Reduction. Step 5 of packages/verifier/src/verify.ts (line 260) explicitly fails with reason binding mismatch unless $C^*_{\mathrm{SNARK}} = C^*_{\mathrm{BBS+}}$, so any winning VP contradicts the assumption that all verifier steps pass. P4 therefore reduces to P1 (cases B and C): $\mathsf{Adv}^{\mathrm{BIND}}_{\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathrm{coll}}_{H_{\mathrm{Pos}}} + \mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathrm{BBS+}}$. The binding argument is for a one-element BBS+ message vector and a single Poseidon commitment; a multi-message extension (e.g. via URDNA2015 canonicalisation) would require a separate canonicalisation argument that the prototype deliberately sidesteps.

F. Out of scope

The argument above is for the abstract construction and does not cover: (i) constant-time or side-channel properties of @digitalbazaar/bbs-signatures or snarkjs; (ii) denial of service, or replay beyond what the BBS+ presentation header provides; (iii) unforgeability under issuer-key compromise: the claims hold only while sk_{BBS} remains honest; (iv) security of the Hermez Powers of Tau ceremony, which is inherited as a 1-of-N-honest assumption; (v) JSON-LD canonicalisation correctness: the prototype's one-element BBS+ message vector intentionally sidesteps URDNA2015; and (vi) the gap between the abstract construction and the running TypeScript code, which would require formal verification of the implementation.

Appendix B
E9 Concurrency Percentile Data

The E9 paragraph in Section VI reports the headline plateau finding; full per-C percentile data is reproduced in Table V for completeness (from evaluation/results/e9_concurrency.summary.json, $n = 60$ timed iterations per C for Groth16, $n = 20$ for PLONK, 5 dropped warm-up). Throughput in presentations per second; latency in milliseconds.

System / C	Tput/s	P50	P95	P99
Groth16, C=1	7.1	140	143	143
Groth16, C=2	7.8	250	280	282
Groth16, C=4	7.8	513	542	551
Groth16, C=8	8.7	907	1,023	1,087
PLONK, C=1	0.44	2,292	2,303	2,303

TABLE V
E9: Groth16 prove+verify under bounded concurrency; PLONK at C=1 for reference. Closed-loop measurement under-reports tail latency under overload (coordinated omission).

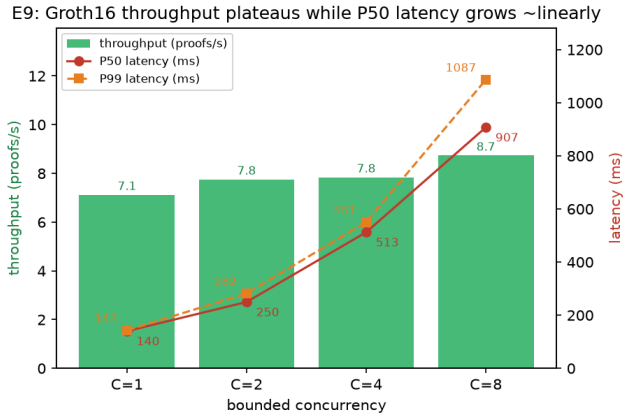


Fig. 3. E9: Groth16 throughput plateaus near 8 presentations/s while P50/P99 latency grows roughly linearly in the bounded concurrency C, which we attribute to single-threaded snarkjs WASM witness generation. Data: evaluation/results/e9_concurrency.summary.json.

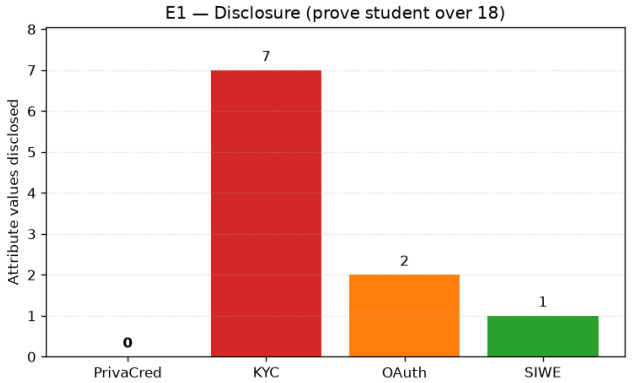


Fig. 4. E1: attribute values disclosed per prove-student-over-18 flow. PrivaCred discloses zero values; baselines disclose between one and seven.

Appendix C
Supplementary Figures and Implementation Table

The figures below plot data already reported numerically in Section VI; they are placed here so the main body carries the findings rather than their restatements. Table VI records the implementation stack referenced in Section V.

Appendix D
Predicate Compiler Benchmark Data

The E8 paragraph in Section VI reports the headline $K \approx 64$ crossover and the Groth16-versus-PLONK prove-side gap; the underlying per-predicate data is reproduced here for completeness (from evaluation/results/e8.csv). Table VII gives six representative demo predicates with their R1CS constraint counts, prove and verify latencies, and proof sizes under both proving systems, alongside the hand-written PrivaCredFullProof baseline. Figure 11 plots the same data.

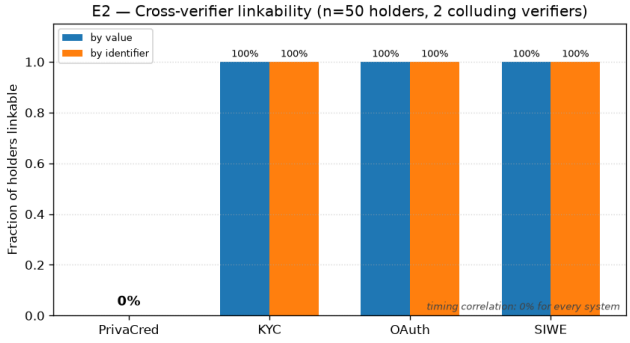


Fig. 5. E2: linkability rates per system per strategy.

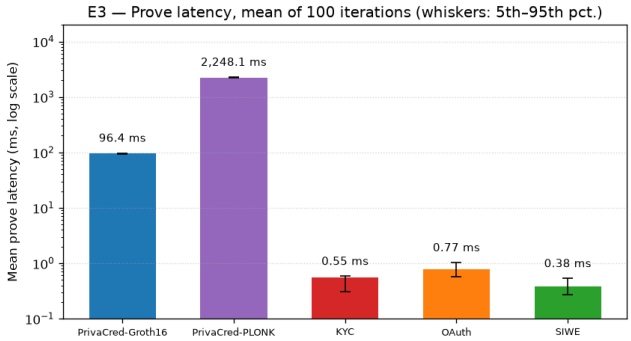


Fig. 6. E3: mean prove-side latency over $n = 100$ iterations, log scale (whiskers: 5th–95th percentile; barely visible because latency is highly stable). The systems span four orders of magnitude.

Appendix E
Implementation Catalogue

The system is a TypeScript (5.4, strict, exactOptional PropertyTypes) monorepo of 11 workspaces (≈13,000 source LOC, counting non-blank, non-comment lines of TypeScript, Circom and JavaScript under packages/ and excluding build output; 189 automated tests passing, one opt-in skipped). Groth16 and PLONK are runtime-selectable over the same R1CS. Table VIII catalogues the workspaces; the source is in the accompanying packages/ archive.

Appendix F
Requirements Verification Matrix

Table IX maps each functional requirement (Section IV) to the contribution that meets it and the experiment that verifies it.

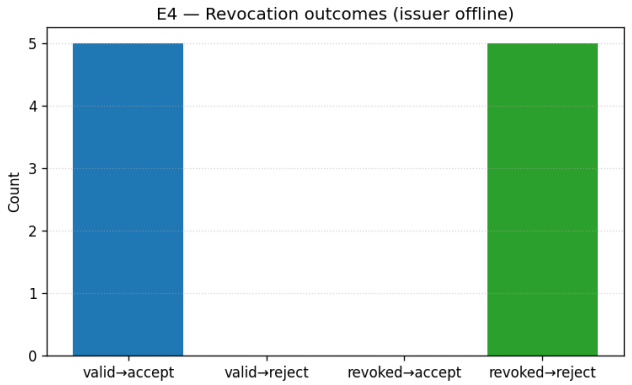


Fig. 7. E4: offline revocation outcomes over the ten-credential grid. Every valid credential is accepted and every revoked one rejected, with no issuer contact at verify time.

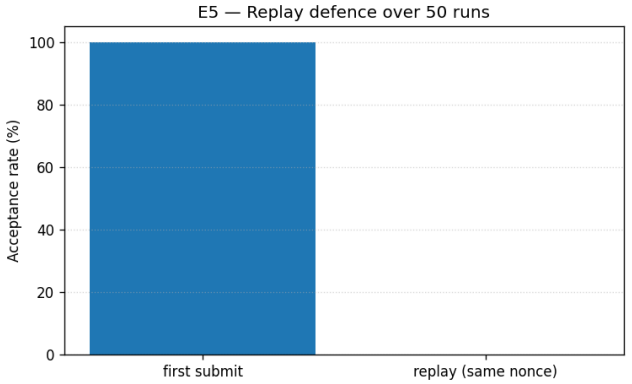


Fig. 8. E5: first-submission acceptance against replay rejection over 50 sessions.

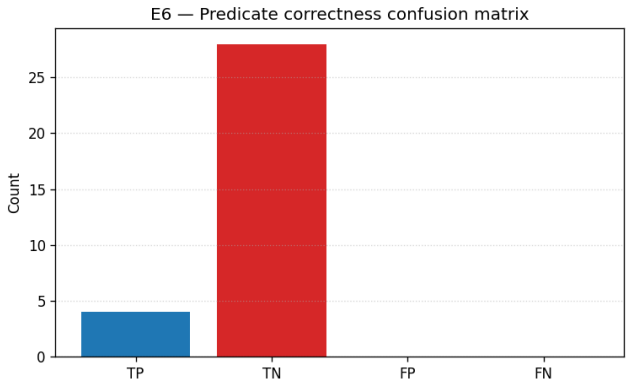


Fig. 9. E6: agreement between the compiled circuit and the independent reference implementation across the predicate boundary grid.

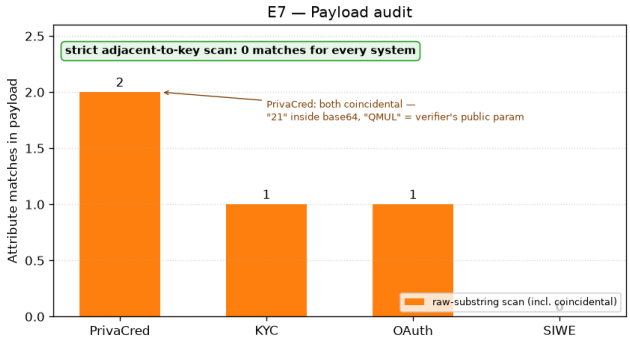


Fig. 10. E7: payload audit under both scans. The strict adjacent-to-key scan (green) is zero for every system. PrivaCred's two raw-substring matches (orange) are coincidental ("21" inside a base64 proof value and the verifier's public "QMUL" parameter, neither a holder attribute), whereas KYC and OAuth expose the holder's email.

TABLE VI
Implementation tech stack.

Layer	Library / Version
Language	TypeScript 5.4 (strict)
Runtime	Node.js 20 LTS
Circuits	circom 2.0
Proving	snarkjs (Groth16 and PLONK)
ZK gadgets	circomlib (Poseidon, comparators)
Off-circuit Poseidon	circomlibjs
BBS+	@digitalbazaar/bbs-signatures
DID	@digitalbazaar/did-method-key
HTTP	Express
SIWE baseline	ethers v6, siwe v2
Testing	Jest, supertest
Evaluation	Python 3.11, NumPy, pandas

TABLE VII

E8: Predicate Compiler benchmark across six representative demo predicates (four AND-only, two OR-of-ANDs; full nine-demo data in evaluation/results/e8.csv). Latencies are means of three iterations on the E3 hardware. Compiler rows report bare snarkjs SNARK verify (∼6 ms); the hand-written baseline row reports the full round-trip verify (BBS+ derived-proof check included, hence ∼47 ms), matching Table II.

Predicate	attrs	R1CS	G16 prove (ms)	G16 verify (ms)	G16 bytes	PLONK prove (ms)	PLONK verify (ms)
student-at-qmul	3	753	109.0	6.9	723	657.8	7.6
alcohol-21-and-country-uk-or-ie	2	630	62.2	6.0	726	354.5	7.4
vaccinated-and-eu-citizen (K=16)	2	688	78.5	5.1	724	376.2	7.4
earns-in-bracket	2	683	64.4	6.0	725	350.3	7.3
OR composition (disjunctive normal form)							
eu-or-uk-citizen (2 clauses)	1	528	60.1	5.8	726	337.7	7.4
student-at-qmul-or-staff-at-qmul (2 clauses)	3	757	67.9	6.4	719	626.7	7.3
hand-written baseline PrivaCredFullProof	3	307	96.4	46.6	—	2,248	48.7

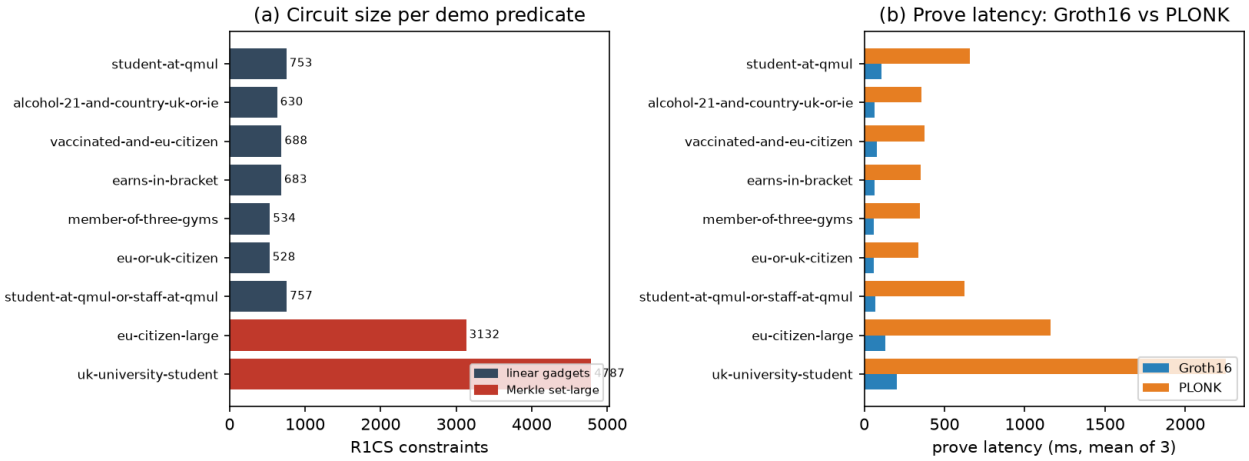


Fig. 11. E8: (a) R1CS constraint count per demo predicate; the two set-large Merkle-of-Poseidon demos (red) dominate, illustrating the $K \approx 64$ crossover finding. (b) Prove-side latency under Groth16 versus PLONK; PLONK is 4.8–11× slower across the demo set. Data: evaluation/results/e8.csv.

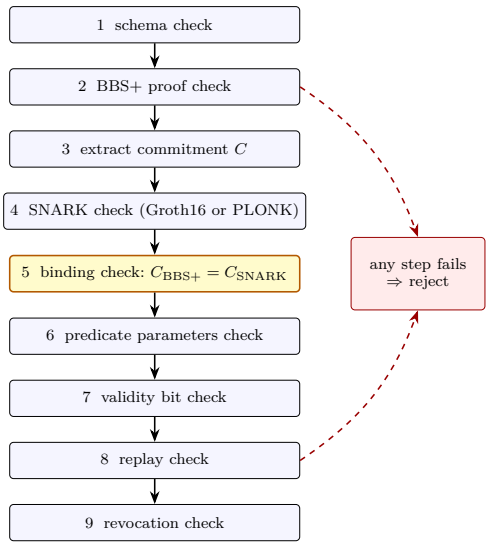


Fig. 12. The verifier's nine-step pipeline (packages/verifier/src/verify.ts). It short-circuits on the first failure; step 5, the binding check, is the heart of Contribution 2.

TABLE VIII

Implementation workspaces.

Workspace	Role
core	shared types; Poseidon, BBS+ envelope, pairwise-DID (HMAC)
issuer	issueCredential, issueCredentialPool, status-list signing
wallet	store; generatePresentation{,ForCircuit,FromPool}; pool helpers
verifier	nine-step pipeline; signed StatusListCache; rotation
circuits	PrivaCredFullProof + Groth16/PLONK setup scripts
compiler	Predicate Compiler (codegen, registry, Merkle set-large)
baseline-kyc	mock KYC reference
baseline-oauth	mock HS256 IdP + real Google RS256/JWKS path
baseline-siwe	Sign-In-With-Ethereum (ethers v6, siwe v2)
demo-server	Express public demonstrator; /share links; deploy.sh
demo-web	Vite browser-side proving (snarkjs WASM)

TABLE IX

Requirements traceability.

Req.	Met by	Verified by
R1 no raw disclosure	commitment + SNARK (§IV)	E1, E7
R2 predicates	circuit + compiler (§IV-D)	E6, E8
R3 layer binding	curve-independent binding (§IV)	verifier step 5; P4
R4 offline verify	cached status list (§V)	E3, E4
R5 revocation	Bitstring Status List (§V)	E4
R6 unlinkability	pairwise DIDs + pool (§IV-E)	E2 (protocol level)
R7 mainstream toolchain	circom/snarkjs (§V)	App. E